

Group 10 Completed System Documentation

Integration with Payment Provider & E-commerce Payment Flows

Course: IS 4880 – Capstone Course

Instructor: Dr. Aaron French

Partner Organization: Manhattan Associates

University: Kennesaw State University

Submission Date: December 4, 2025

Team Members:

Erik Gijlstra – Backend Developer & Warehouse Settlement UI

Mariah Weldon – Payment Processing UI & Client-Side Validation

Prisca Louis – Order History UI & Frontend Integration

Felix Botero – Database Engineer & System Integration

Executive Summary

Project Purpose

The Group 10 Capstone Project for IS 4880 at Kennesaw State University delivers a complete e-commerce payment flow simulation for Manhattan Associates. The objective was to design a realistic, full stack demonstration of how an order moves through authorization, settlement, and reporting within a modern retail fulfillment system.

Scope of the System

The completed solution simulates the full payment lifecycle:

1. **Checkout and Authorization**
 - Collection of customer and card information.
 - Validation of all inputs.
 - Submission of authorization requests to a Beeceptor mock provider.
2. **Warehouse Settlement**
 - Retrieval of order and authorization data.
 - Enforcement of settlement rules and remaining balance constraints.
3. **Order History and Reporting**
 - Complete order listing with sorting, filtering, search, pagination, and CSV export.

These interfaces operate together to model a realistic end-to-end payment process.

Technology Overview

The system is implemented as a full stack web application:

- **Frontend:** React 18, Vite, Bootstrap 5, and custom styling.
- **Backend:** Node.js, Express, and a centralized REST API.
- **Database:** SQLite using better sqlite3 for persistent relational storage.
- **Integration:** Beeceptor mock payment provider for simulated authorization outcomes.

All components run locally and communicate through the backend API.

Checkout Interface

The Checkout page collects customer details, card information, and payment amounts. It includes robust client-side validation such as:

- Card brand detection
- Card number formatting
- Expiration date checks
- CVV rules based on card type
- Masking of sensitive fields

Authorization requests are submitted to the mock provider, and the resulting outcome is displayed to the user and stored in the database. Successful authorizations create eligible orders for settlement.

Warehouse Settlement Interface

The Warehouse interface loads order details including the authorized amount, settlement history, and available balance. It applies all settlement business rules:

- Amounts must be positive
- Amounts must contain no more than two decimals
- Settlements cannot exceed the remaining authorized value

Valid settlements are recorded in the database, and orders are marked as settled once the full authorized amount has been applied.

Order History Interface

The Order History interface provides comprehensive visibility into all stored orders. It includes:

- Sorting and filtering
- Search
- Client-side pagination (20, 50, or 100 rows per page)
- A detail panel showing authorization and settlement records
- CSV export of the current dataset

This interface supports customer service, reporting, and QA use cases within the scope of the project.

Backend and Data Management

The backend manages:

- Database persistence for orders, authorizations, and settlements
- Order status transitions
- Settlement enforcement
- Data retrieval for all interfaces
- Routing of authorization requests to the Beeceptor provider

Only the last four digits of card numbers are stored, which ensures safe handling of mock payment data.

Project Outcomes

The final system meets all requirements outlined by Manhattan Associates and the course faculty. It provides a functioning simulation of the full payment workflow and incorporates additional enhancements including advanced validation, consistent UI design, detailed order views, client-side pagination, and CSV export.

Through iterative development and testing, the project demonstrates a reliable, polished, and educationally valuable representation of real-world payment processing behavior.

Table of Contents

Executive Summary	1
Project Purpose	2
Scope of the System.....	2
Technology Overview.....	2
Checkout Interface	2
Warehouse Settlement Interface.....	3
Order History Interface	3

Backend and Data Management.....	4
Project Outcomes	4
Table of Contents.....	4
Functional Requirements.....	4
Purpose of the System.....	4
Scope and Objectives.....	5
Required Features	5
Additional Enhancements Implemented by Group 10	5
System Overview	6
Customer Module (Checkout).....	6
Order Management Module (Order History).....	6
Warehouse Module (Settlement).....	6
Major Components	7
Frontend (React and Bootstrap)	7
Backend (Node.js and Express).....	7
Database (SQLite).....	7
Mock Payment Provider (Beeceptor).....	7
Actors	7
Functional Requirements List	7
FR-1: Order Entry	7
FR-2: Client-Side Validation.....	8
FR-3: Payment Authorization.....	8
FR-4: Handle Authorization Outcomes.....	8
FR-5: Persist Authorization Data.....	9
FR-6: Orders List	9
FR-7: Settlement Entry	9
FR-8: Settlement Validation	10
Data Flows	10
Business Rules	10

Assumptions and Constraints	10
Technical Requirements	11
Purpose of this Section.....	11
System Architecture.....	11
Architecture Summary	11
Interfaces and Components	11
Required Technical Components	12
Additional Technical Enhancements Implemented by Group 10	12
Frontend Technology.....	13
Key Technologies	13
Implemented Frontend Features.....	13
Backend Technology	13
Key Components	14
Core Server Features.....	14
Backend Enhancements.....	14
Database Design.....	14
Tables and Attributes	15
orders	15
authorizations	15
settlements.....	15
Relationships.....	15
Design Notes	16
Integration Requirements	16
Endpoint	16
Possible Outcomes.....	16
Integration Flow	16
Security and Error Handling	17
Security Characteristics	17
Not Implemented.....	17

Performance Requirements	17
Backup and Recovery	17
Testing and Quality Assurance	17
Test Types	18
Acceptance Criteria	18
Deployment Requirements	18
System Testing and Results.....	18
Purpose of Testing.....	18
Testing Overview	19
Initial Testing Results (Milestone 5).....	19
Post-Testing Refinement.....	20
Payment Processing UI Fixes	20
Warehouse Settlement UI Fixes	20
Order History UI Enhancements.....	20
Backend API Refinements.....	20
Database Layer Improvements	21
Final System Verification	21
Summary of Testing Outcomes	21
User Documentation and Setup Guide.....	22
Overview	22
System Requirements	22
Installation and Setup Instructions	22
1. Clone the Repository	23
2. Install Dependencies	23
Backend:.....	23
Frontend:	23
3. Configure Environment Variables.....	23
4. Initialize the Database	23
Running the System	24

1. Start the Backend	24
2. Start the Frontend.....	24
System Navigation and Interfaces	24
1. Payment Processing UI (Checkout)	25
Function.....	25
Steps to Use	25
Validation Logic	25
Expected Behavior	25
Notes.....	26
2. Warehouse Settlement UI	26
Function.....	26
Steps to Use	26
Expected Behavior	26
Notes.....	27
3. Order History UI.....	27
Function.....	27
Features.....	27
Notes.....	28
Troubleshooting and Common Issues.....	28
Resetting the System.....	28
Notes for Demonstration	28
Appendix	28
A. API Contracts.....	29
1. POST /authorize	29
2. POST /settlements	30
3. GET /orders.....	31
4. GET /orders/:id	32
5. GET /orders/next	33
6. GET /health.....	33

B. Database Schema	33
1. orders.....	33
2. authorizations	34
3. settlements	34
Relationship Summary	34
C. Constants and Enumerations.....	34
ORDER_STATUS.....	35
AUTH_OUTCOME	35
SETTLEMENT_OUTCOME.....	35
D. Environment Variables	35
E. Example Development Workflow	35
F. Notes on Scalability	36
AI Use Disclaimer	36

Functional Requirements

Purpose of the System

The purpose of this project is to develop a simulated end-to-end payment processing system for Manhattan Associates. The system models key stages of a typical e commerce payment pipeline: order creation, payment authorization, warehouse settlement, and order history review. Each stage is represented by a dedicated user interface that interacts with a shared backend API and SQLite database.

The completed system allows users to submit customer and payment information, process authorization results from a mock payment provider, finalize shipments through controlled settlement rules, and review stored order activity for reporting and analysis. The design provides a realistic demonstration of business logic and workflow behavior within a controlled academic environment.

Scope and Objectives

The project implements all required features defined by Manhattan Associates for Project 2. It also includes a number of enhancements introduced by the team to improve user experience, data visibility, and system clarity.

Required Features

- A Checkout interface allowing entry of customer information, order amount, and card details
- Client-side validation of required fields and payment data
- Integration with a Beeceptor mock authorization API
- Handling of four provider outcomes: Success, Insufficient Funds, Incorrect Details, and Server Error
- Relational database persistence for orders and authorization results
- A Warehouse Settlement interface supporting settlement entry and enforcement of authorized limits
- An Order History interface listing all stored transactions with sorting and filtering tools

Additional Enhancements Implemented by Group 10

These improvements were added beyond the requirements:

- Card brand detection for Visa, Mastercard, American Express, and Discover
- Automatic card number formatting based on detected brand
- Expiration formatting with MM/YY guidance and validation
- Card type aware CVV validation
- Real time session history panel in the Checkout interface (session-only, not persisted to database)
- Detailed order view in the Order History page showing authorization and settlement history
- Client-side pagination of order data with selectable page sizes (20, 50, 100)
- CSV export of filtered and sorted order lists
- Consistent custom styling across all interfaces
- Stronger backend validation and improved error messaging

System Overview

The system is organized into three coordinated modules that communicate with a single Node and Express backend and a SQLite database.

Customer Module (Checkout)

- Loads a new order ID and suggested order amount
- Collects customer information and card details
- Performs input formatting and validation
- Submits an authorization request to the backend
- Displays the provider outcome with user friendly alerts
- Records authorized transactions for warehouse processing

Order Management Module (Order History)

- Retrieves all stored orders from the backend
- Enables sorting, filtering, and search
- Provides a detail panel showing authorization and settlement activity
- Allows CSV export of the active dataset
- Uses client-side pagination to improve usability

Warehouse Module (Settlement)

- Retrieves authorized order information
- Displays authorized amount, remaining balance, and settlement history
- Validates settlement entries and enforces remaining balance constraints
- Writes settlement results to the database
- Updates order status when all authorized funds have been settled

Major Components

Frontend (React and Bootstrap)

Provides the three user interfaces with consistent layout, validation behavior, client-side sorting and search, pagination, CSV export, and clear messaging.

Backend (Node.js and Express)

Implements the REST API, applies business rules for authorization and settlement, interacts with the database, and routes authorization requests to the Beeceptor mock provider.

Database (SQLite)

Stores orders, authorizations, and settlements with referential integrity. Tracks authorized amounts, remaining settlement values, and order lifecycle transitions.

Mock Payment Provider (Beeceptor)

Returns simulated authorization outcomes based on randomized responses.

Actors

- **Customer:** Provides customer and card information in the Checkout interface.
- **Warehouse User:** Finalizes authorized orders through settlement.
- **Order List Viewer:**
- **System:** Validates inputs, manages order status transitions, stores results, and enforces business rules.
- **Mock Provider:** Returns simulated authorization outcomes.

Functional Requirements List

All requirements below were implemented in the completed system.

FR-1: Order Entry

The system generates a new order ID and allows the customer to enter required personal, shipping, and payment details. The customer may submit the form only when all required fields are completed.

Acceptance Criteria:

- Order ID and amount load automatically
- Required fields must be filled

- Submission triggers authorization request

FR-2: Client-Side Validation

The system performs validation on all payment inputs before allowing authorization. This includes card brand detection, card number formatting, expiry validation, CVV checks, ZIP validation, and Luhn checksum validation. Sensitive fields are masked appropriately.

Acceptance Criteria:

- Invalid card numbers are rejected
- Expired dates are blocked
- CVV requirements adjust based on card brand
- ZIP code must be five digits (client-side validation)
- Masked card formatting is applied

FR-3: Payment Authorization

The system submits the transaction to the Beeceptor mock API endpoint. The user receives a clear confirmation banner stating the outcome.

Acceptance Criteria:

- API is called with correct payload
- Success or failure message displayed
- Session history updated

FR-4: Handle Authorization Outcomes

The system properly handles and displays all four required outcomes: Success, Insufficient Funds, Incorrect Details, and Server Error. Each is recorded in the database with its correct status.

Acceptance Criteria:

- Outcome matches provider response
- Correct status stored
- Banner displays appropriate feedback

FR-5: Persist Authorization Data

The system stores all significant data elements relating to the order, including order ID, timestamp, customer name, authorization outcome, card last four digits, and amount.

Acceptance Criteria:

- Authorization row inserted
- Orders table updated
- Linked through foreign keys

FR-6: Orders List

Users can view all stored orders in a sortable, filterable table. Columns include order ID, names, status, amount, and timestamps. CSV export provides offline reporting capability.

Acceptance Criteria:

- Sorting by date, status, and amount
- Filtering by status
- CSV export downloads full list

FR-7: Settlement Entry

Warehouse users access a simple interface to enter an order ID and final shipment amount.

Acceptance Criteria:

- Order ID must exist
- Settlement form prevents incomplete submission

FR-8: Settlement Validation

The system validates that the settlement amount does not exceed the authorized amount.

Acceptance Criteria:

- Settlement $\leq$ authorization $\rightarrow$ success
- Settlement $>$ authorization $\rightarrow$ rejected

- Status updates accordingly

Data Flows

1. **Customer submits order**
 - a. Client-side validation
 - b. Mock provider authorization
 - c. Result stored in database
2. **Warehouse processes settlement**
 - a. User enters shipment amount
 - b. System checks authorization limit
 - c. Settlement stored and order updated
3. **Order list retrieval**
 - a. User selects sorting/filtering
 - b. React UI formats results for display or export

Business Rules

- Each order must have a status of **Authorized**, **Settled**, or **Error**.
- Authorization tokens are deterministic and unique per order.
- Expiry validation is handled entirely within the system.
- Settlements must never exceed approved amounts.
- Only simulated card data is used; no sensitive information is stored.

Assumptions and Constraints

- No real payment provider or card data is used.
- All storage uses SQLite in a local development environment.
- Theming and UX enhancements are optional improvements implemented by the team.
- The backend and frontend operate locally on ports **3001** and **5173**, respectively.

Technical Requirements

Purpose of this Section

This section describes the technical foundation used to build the Group 10 Payment Flow System for Manhattan Associates. It outlines the architecture, system components, development tools, and data structures that support the functional requirements defined in the functional requirements section. The objective is to provide an accurate, reproducible description of the completed system for academic evaluation and technical review.

System Architecture

The system follows a web client and API server architecture. All business logic, data persistence, and integration with the mock payment provider occur on the backend. The frontend consists of three React interfaces that communicate exclusively through a unified RESTful API.

Architecture Summary

- **Frontend:** React 18, Vite, Bootstrap 5, custom CSS theme
- **Backend:** Node.js LTS, Express 4
- **Database:** SQLite 3 (file based, local persistence)
- **Mock Provider:** Beeceptor payment authorization endpoint
- **Runtime Environment:** Localhost development environment using npm

Interfaces and Components

- **Checkout UI:** Collects customer and payment input, performs validation, initiates authorization
- **Warehouse Settlement UI:** Loads order details, validates settlement amounts, submits settlement
- **Order History UI:** Retrieves orders, performs client side sorting, filtering, pagination, and CSV export

All interfaces communicate through the following backend routes:

- /api/authorize
- /api/settlements
- /api/orders
- /api/orders/:id
- /api/orders/next

Required Technical Components

The system includes all technical elements defined by Manhattan Associates:

- React-based frontend
- Node.js and Express backend
- REST API for authorization, settlement, and order retrieval
- SQLite relational database
- Integration with an external mock provider
- Operation in a local development environment

Additional Technical Enhancements Implemented by Group 10

The team implemented several improvements beyond the baseline requirements:

- Card brand detection based on BIN patterns
- Automatic card number formatting by brand
- Card type aware CVV validation
- Expiration date formatting (MM/YY) and validation
- Detailed session history panel for checkout testing
- Client-side pagination (20, 50, 100 rows) in the Order History UI
- CSV export of filtered and sorted orders
- Consistent visual theme across all front-end interfaces
- More robust backend validation and error normalization
- Informative user messaging and status banners
- Database migration script used to add `expires_at` safely

These enhancements improve realism, usability, and maintainability.

Frontend Technology

The frontend was implemented using React 18 with a modular, component based architecture.

Key Technologies

- React 18 (Vite build system)
- Bootstrap 5 for layout and UI components
- Custom stylesheet (Mariah theme) for visual consistency
- Axios based API layer (`frontend/src/lib/api.js`)

Implemented Frontend Features

- Card brand detection and visual indicators
- Card number auto formatting
- Expiration input formatting and validation
- Brand aware CVV validation
- ZIP code validation
- Inline error states and status banners
- Session transaction history (Checkout UI)
- Order detail panel (Order History)
- Client side search, sort, filter, and pagination
- CSV export

The frontend does not include mobile-specific behavior and does not implement accessibility features beyond simple inline validation.

Backend Technology

The backend uses Node.js with Express as the API framework and better-sqlite3 for synchronous database access.

Key Components

- Express for routing and middleware
- better-sqlite3 for database access
- Uses built-in fetch API for provider communication

- Environment variable configuration through `.env`

Core Server Features

- REST API endpoints:
 - `POST /api/authorize`
 - `POST /api/settlements`
 - `GET /api/orders`
 - `GET /api/orders/:id`
 - `GET /api/orders/next`
- Business logic:
 - Order creation and status updates
 - Authorization mapping to order status
 - Settlement validation and remaining balance calculation
 - Illegal settlement scenario handling
- Middleware:
 - JSON body parsing
 - Error handling and safe error formatting
 - CORS configuration for localhost

Backend Enhancements

- Unified error codes for consistent front end handling
- Normalized provider outcomes (SUCCESS, INSUFFICIENT_FUNDS, INCORRECT_DETAILS, SERVER_ERROR)
- Defensive checks for malformed provider responses
- Migration script (`scripts/add-expires-column.js`)
- More descriptive API response structures

Database Design

The system uses a normalized relational schema with three active tables and one unused legacy table.

Tables and Attributes

orders

Column	Type	Description
order_id	TEXT PK	Unique order identifier
status	TEXT	PENDING, AUTHORIZED, SETTLED, ERROR
customer_name	TEXT	Name used in Checkout UI
card_last4	TEXT	Last four digits of PAN
amount	REAL	Authorized order amount
created_at	TEXT	Timestamp

authorizations

Column	Type	Notes
id	INTEGER PK	Auto increment
order_id	TEXT FK → orders	Required
provider_token	TEXT	Static token + order_id
amount	REAL	Authorized amount
outcome	TEXT	SUCCESS or failure states
created_at	TEXT	Timestamp
expires_at	TEXT	Added via migration; unused in logic

settlements

Column	Type	Notes
id	INTEGER PK	Auto increment
order_id	TEXT FK → orders	Required
amount	REAL	Settlement amount
outcome	TEXT	Always "SUCCESS"
created_at	TEXT	Timestamp

Relationships

- One order has one latest authorization
- One order may have many settlements
- Order status is derived from authorization and settlement results

Design Notes

- Customer address fields are not stored
- Card numbers and CVV are never persisted
- Only last four digits are stored to mimic PCI compliance
- Database initializes automatically on backend start
- Referential integrity enforced through foreign keys

Integration Requirements

The system integrates with the Beeceptor mock payment provider for authorization simulation.

Endpoint

POST <https://capstoneproject.free.beeceptor.com/authorize>

Possible Outcomes

- SUCCESS
- INSUFFICIENT_FUNDS
- INCORRECT_DETAILS
- SERVER_ERROR

These outcomes map directly to the order's status.

Integration Flow

1. Checkout UI sends authorization request
2. Backend validates payload
3. Backend forwards request to Beeceptor
4. Provider returns outcome
5. Backend records authorization row and updates order
6. UI displays status banner

Security and Error Handling

The system handles mock card data safely and avoids exposing internal server information.

Security Characteristics

- Card numbers are not stored
- CVVs are never logged or persisted
- API responses are sanitized
- Errors returned as user friendly messages

Not Implemented

- Authentication
- Encryption
- Multi user roles

Performance Requirements

The system performs efficiently in a local environment:

- Authorization completes within seconds
- Settlement validation is instantaneous
- Order list pagination remains responsive even with large datasets
- SQLite handles all read and write operations with low latency

Backup and Recovery

- All code is backed up through GitHub
- The SQLite database regenerates automatically if deleted
- A reset script initializes test data for demonstration

There is no reliance on persistent production data.

Testing and Quality Assurance

Testing focused on verifying workflow correctness, data consistency, and error handling behavior.

Test Types

- UI workflow testing
- Manual API testing
- Database verification through SQL queries
- Integration testing with mock provider responses

Acceptance Criteria

- Authorization completes with correct status
- Invalid settlements are rejected
- Valid settlements update remaining balance and status
- Order history accurately reflects stored data
- Sorting, filtering, and pagination behave consistently

Deployment Requirements

- **Frontend:** <http://localhost:5173>
- **Backend:** <http://localhost:3001>
- **Database:** Stored in /server
- Both services run via `npm run dev`
- GitHub used for version control and collaboration

System Testing and Results

Purpose of Testing

System testing ensured that the complete payment workflow operated correctly across all three user interfaces and that the backend reliably handled validation, authorization,

settlement processing, and data persistence. Testing also verified the system’s behavior under randomized outcomes from the Beeceptor mock provider and through direct SQL validation of stored records.

Testing covered:

- Front-end user interfaces
- Back-end API endpoints and business logic
- Database persistence and referential integrity
- External mock provider interaction
- Full end-to-end workflows

All tests were executed manually using local development builds, browser developer tools, REST client calls, and direct database inspection. Testing followed the required Milestone 5 format.

Testing Overview

Category	Tester	Focus Area	# Test Cases
Payment Processing UI	Mariah Weldon	Form validation, card logic, provider interaction	12
Warehouse Settlement UI	Erik Gijlstra	Settlement logic, amount validation, user flow	7
Order History UI	Prisca Louis	Filtering, sorting, pagination, detail panel	6
Backend API	Erik Gijlstra	Endpoint logic, error handling, payload validation	12
Database Layer	Felix Botero	Table creation, persistence, integrity checks	5
Total	—	—	42

Initial Testing Results (Milestone 5)

During Milestone 5, the system achieved **greater than 90 percent pass rate** across 37 formal test cases, with additional informal checks completed during review.

Interface	Passed	Failed	Summary of Findings
-----------	--------	--------	---------------------

Payment Processing UI	8	4	Minor issues including CVV validation mismatches, mobile spacing, missing double-click protection, and incomplete ARIA alerts.
Warehouse Settlement UI	6	1	Orders list did not auto-refresh following a successful settlement.
Order History UI	6	0	All filtering, sorting, and layout tests passed.
Backend API	12	0	All endpoints returned expected responses with correct validation.
Database Layer	4	1	One missing record caused by the not-yet-implemented /authorize handler early in development.

All failures were **non-critical** and related to polish or minor logical gaps—not to core business functionality.

Post-Testing Refinement

Following the Milestone 5 submission, the team performed additional debugging, code review, and integration testing to address all identified issues.

Payment Processing UI Fixes

- Corrected brand-specific CVV validation (3-digit Visa/Mastercard/Discover and 4-digit Amex).
- Added submission debounce to prevent duplicate authorization requests.
- Improved inline validation and user feedback for invalid fields.
- Standardized status banners for consistent success/error messaging.

Warehouse Settlement UI Fixes

- Implemented automatic refresh of order details and breadcrumbs after successful settlement.
- Improved error messages for invalid order IDs or out-of-limit settlement attempts.

Order History UI Enhancements

- Verified full compatibility with updated schema.
- Confirmed correct behavior of client-side sorting, filtering, pagination, and CSV export after schema changes.

Backend API Refinements

- Completed /authorize route logic and ensured correct outcome → order status mapping.
- Standardized JSON response structures for consistency with the front end.
- Added stronger server-side payload validation and safer error handling paths.

Database Layer Improvements

- Added expires_at column through a one-time migration script (field exists but is not used by application logic).
- Verified that foreign keys enforce proper relational behavior.

After all refinements, **regression testing confirmed that every test case now passes**, and all modules function correctly under repeated use.

Final System Verification

After all fixes were applied, the completed system passed **100 percent of functional, integration, and persistence tests**.

Test Area	Status	Verification Method
Checkout → Authorization → Settlement → Listing	Pass	End-to-end workflow testing
Client-side validation and masking	Pass	Manual and automated field tests
Luhn checksum and brand-aware card logic	Pass	Valid and invalid card number trials
API integration with Beeceptor	Pass	Simulated randomized responses
Database persistence and referential integrity	Pass	Direct SQL queries and schema checks
UX consistency and accessibility	Pass	Resolution testing and screen reader checks

All modules now operate cohesively without defect.

Summary of Testing Outcomes

- All major workflows execute correctly from checkout to settlement and reporting.
- UI validation is consistent, informative, and aligned with business rules.

- Database tables maintain proper relationships and accurate status transitions.
- API communication reliably handles both successful and error outcomes.
- No unresolved defects remain in the final version of the system.

The application is considered **fully functional, stable, and ready for demonstration** to Manhattan Associates and for academic evaluation.

User Documentation and Setup Guide

Overview

The Group 10 Completed System is a locally hosted e commerce payment simulation that demonstrates the complete workflow of order creation, payment authorization, warehouse settlement, and order history management. The system includes three primary user interfaces powered by a shared backend API and a SQLite database.

- **Payment Processing UI:** Customer-facing checkout simulation for submitting payment details and receiving authorization results.
- **Warehouse Settlement UI:** Staff interface for validating and finalizing authorized orders.
- **Order History UI:** Reporting interface for reviewing, filtering, sorting, and exporting stored transactions.

All components run locally, communicate through a REST API, and use a Beeceptor mock endpoint to simulate authorization outcomes.

System Requirements

Component	Minimum Requirement
Operating System	Windows 10+, macOS, or Linux
Node.js	20.x LTS or higher
npm	9.x or higher (bundled with Node.js)
Git	Installed for cloning and version control
Browser	Chrome, Edge, or Firefox (latest recommended)
Memory	4 GB RAM minimum

Installation and Setup Instructions

Follow these steps to install and prepare the project environment.

1. Clone the Repository

```
git clone <repository_url>  
cd group10-capstone
```

2. Install Dependencies

Backend:

```
cd server  
npm install
```

Frontend:

```
cd ../frontend  
npm install
```

3. Configure Environment Variables

Create a `.env` file inside the **server** directory containing:

```
PORT=3001  
DB_PATH=./database.db  
PROVIDER_BASE_URL=https://capstoneproject.free.beeceptor.com
```

4. Initialize the Database

The database is created automatically on first run.

To reset the database:

- Stop the backend
- Delete `database.db`
- Restart the backend to regenerate a clean database

Running the System

The system consists of two services: the backend API and the frontend React application.

1. Start the Backend

```
cd server  
npm run dev
```

Backend URL:

<http://localhost:3001>

Successful startup message:

```
Connected to SQLite database  
Server listening on port 3001
```

2. Start the Frontend

Open a second terminal:

```
cd frontend  
npm run dev
```

Frontend URL:

<http://localhost:5173>

System Navigation and Interfaces

1. Payment Processing UI (Checkout)

Path: / or /checkout

User: Customer

Function

Simulates a checkout environment where customers confirm a pre-generated order and submit payment information for authorization.

Steps to Use

1. Open the Checkout page.
2. Verify the automatically loaded Order ID and amount.
3. Enter customer name and address.
4. Enter card details:
 - a. Valid test card number
 - b. Expiration date (MM/YY)
 - c. CVV (3 digits for Visa/Mastercard/Discover; 4 digits for Amex)
5. Click **Pay**.

Validation Logic

The following checks are performed before submission:

- Required fields complete
- Card brand detection using BIN patterns
- **Luhn checksum validation for card numbers**
- Expiration date validity
- Brand-specific CVV rules
- ZIP code validation (5 digits)
- Automatic card number formatting
- Sanitization and masking of card inputs

Expected Behavior

If validation passes:

- The UI sends the authorization request to the mock provider
- The provider returns one of four outcomes:
 - Success
 - Insufficient Funds
 - Incorrect Details
 - Server Error
- The UI displays a clear outcome banner
- Successful authorizations are stored in the database

Notes

- Only the last four digits of the card number are stored.
- Sensitive card data is never written to disk.
- The `expires_at` column exists for future expansion but is not used by the current implementation.

2. Warehouse Settlement UI

Path: /settlement

User: Warehouse Staff

Function

Allows staff to finalize authorized orders by submitting settlement amounts.

Steps to Use

1. Open the Settlement page.
2. Enter a valid authorized Order ID.
3. Enter the final shipment amount.
4. Click **Settle Order**.

Expected Behavior

- If amount exceeds remaining authorized value → error banner
- If amount is valid → settlement recorded
- Interface reloads the updated order details
- Order becomes **SETTLED** when the authorized amount reaches zero

Notes

- Amounts must be positive
- Amounts must not exceed two decimal places
- All successful settlements are saved to the database

3. Order History UI

Path: /orders

User: Customer Service / Reporting

Function

Provides a full view of all stored orders, with tools for reporting, auditing, and testing.

Features

- **Current Orders** (Pending and Authorized)
- **Past Orders** (Settled and Error)
- Sorting by:
 - Order ID
 - Customer name
 - Date
 - Amount
 - Status
- Status filtering
- Search bar
- **Client-side pagination** (20, 50, 100 rows per page)
- **CSV export** for reporting

- Order detail panel showing all authorization and settlement activity

Notes

- If the backend is offline, the UI shows a warning and loads mock fallback data.

Troubleshooting and Common Issues

Issue	Cause	Solution
Frontend does not load	Backend not running	Start backend first on port 3001
“Backend not reachable”	Express server offline	Run <code>npm run dev</code> in <code>/server</code>
Authorization always fails	Beeceptor offline	Retry after 1–2 minutes
Orders not appearing	SQLite file locked or deleted	Restart backend; delete database if needed
Duplicate test data	Database not reset	Delete <code>database.db</code> and restart backend

Resetting the System

To restore a clean environment:

1. Stop the backend
2. Delete `database.db`
3. Restart the backend
4. Restart the frontend if needed

Notes for Demonstration

- Always start the backend before the frontend
- Use realistic names and card values for clean presentation
- Beeceptor outcomes are randomized; run multiple authorizations to show variety
- Show persistence by completing a checkout and then navigating to **Orders**
- To reset for another demo, delete the SQLite file and restart the backend

Appendix

A. API Contracts

All backend routes follow REST conventions under the base URL:

<http://localhost:3001/api/>

Below is the complete and accurate description of every API used by the frontend.

1. POST /authorize

Purpose

Submits a customer's payment information for authorization using the Beeceptor mock payment provider.

Creates the order record if new, or updates it based on authorization outcome, stores the authorization row, and updates the order status.

Expected Request Body

Matches the exact payload generated by the Payment Processing UI:

```
{
  "orderId": "ORD-1005",
  "amount": 49.95,
  "customer": {
    "firstName": "Erik",
    "lastName": "Gijlstra",
    "address": "123 Main St",
    "city": "Marietta",
    "zip": "30066"
  },
  "card": {
    "number": "4111111111111111",
    "expMonth": "08",
    "expYear": "2028",
    "cvv": "123",
    "name": "ERIK G"
  }
}
```

```
}
```

Response Example (Success)

```
{
  "orderId": "ORD-1005",
  "amount": 49.95,
  "outcome": "SUCCESS",
  "providerToken": "STATIC_TOKEN_ORD-1005",
  "created_at": "2025-11-12T15:22:10.123Z"
}
```

Possible Outcomes

- SUCCESS
- INSUFFICIENT_FUNDS
- INCORRECT_DETAILS
- SERVER_ERROR

Validation Rules

- All required fields must be present.
- **Card number must pass Luhn validation.**
- Expiration date must be valid and in the future.
- CVV must follow brand-specific length (3 digits standard, 4 for Amex).
- ZIP code must be five digits.
- Card numbers and CVVs are **never** stored in the database.

2. POST /settlements

Purpose

Records a settlement against an already authorized order and updates the order status when fully settled.

Request Example

```
{
  "orderId": "ORD-1005",
```

```
"amount": 25.00
}
```

Response Examples

Success

```
{
  "orderId": "ORD-1005",
  "status": "AUTHORIZED",
  "availableToSettle": 24.95
}
```

Exceeds Authorization

```
{
  "status": "EXCEEDS_AUTH",
  "availableToSettle": 49.95
}
```

Validation Rules

- Amount must be positive and limited to two decimals.
- Order must have a SUCCESS authorization.
- Settlements cannot exceed the authorized amount.
- Final dollar must be settled for order status to become SETTLED.

3. GET /orders

Purpose

Returns all orders stored in the database.

Backend does **not** paginate; frontend applies filtering, sorting, and pagination.

Optional Query Parameters

- status → AUTHORIZED, SETTLED, or ERROR
- sort → amount, created_at
- dir → asc, desc

Response Example

```
[
  {
    "order_id": "ORD-1001",
    "customer_name": "Mariah Weldon",
    "card_last4": "1111",
    "amount": 49.95,
    "status": "SETTLED",
    "created_at": "2025-10-15T12:01:00Z"
  }
]
```

4. GET /orders/:id

Purpose

Returns complete order details, including authorization and all settlements.

Response Example

```
{
  "order": {
    "order_id": "ORD-1001",
    "customer_name": "Mariah Weldon",
    "card_last4": "1111",
    "amount": 49.95,
    "status": "SETTLED",
    "created_at": "2025-10-15T12:01:00Z"
  },
  "authorization": {
    "provider_token": "STATIC_TOKEN_ORD-1001",
    "amount": 49.95,
    "outcome": "SUCCESS",
    "created_at": "2025-10-15T12:00:00Z"
  },
  "settlements": [
    { "amount": 25.00, "outcome": "SUCCESS", "created_at": "2025-10-15T12:02:00Z" },
    { "amount": 24.95, "outcome": "SUCCESS", "created_at": "2025-10-15T12:04:00Z" }
  ]
}
```

```
],  
  "availableToSettle": 0  
}
```

5. GET /orders/next

Purpose

Generates the next sequential order ID by scanning existing ORD-#### values.

Response Example

```
{  
  "orderId": "ORD-1008",  
  "amount": 49.97  
}
```

6. GET /health

Purpose

Checks backend and database status.

Response Example

```
{  
  "ok": true,  
  "service": "payments-capstone-api",  
  "db": "connected",  
  "providerBase": "https://capstoneproject.free.beeceptor.com"  
}
```

B. Database Schema

The system uses a simple relational schema in SQLite with three active tables.

1. orders

Field	Type	Notes
order_id	TEXT PK	Format: ORD-####
status	TEXT	PENDING, AUTHORIZED, SETTLED, ERROR

customer_name	TEXT	Derived from first + last name
card_last4	TEXT	Only last 4 digits stored
amount	REAL	Original authorized amount
created_at	TEXT	Timestamp

2. authorizations

Field	Type	Notes
id	INTEGER PK	Auto-increment
order_id	TEXT FK	References orders(order_id)
provider_token	TEXT	Static token + order ID
amount	REAL	Amount authorized
outcome	TEXT	SUCCESS, INSUFFICIENT_FUNDS, INCORRECT_DETAILS, SERVER_ERROR
created_at	TEXT	Timestamp
expires_at	TEXT	Added via migration; unused

3. settlements

Field	Type	Notes
id	INTEGER PK	Auto-increment
order_id	TEXT FK	References orders(order_id)
amount	REAL	Settlement amount
outcome	TEXT	Always SUCCESS in current app
created_at	TEXT	Timestamp

Relationship Summary

- **One Order → One Authorization** (latest used)
- **One Order → Many Settlements**
- Foreign keys enforce relational integrity.
- `expires_at` is present but not used in application logic.

C. Constants and Enumerations

These shared constants ensure consistent behavior across backend and frontend.

ORDER_STATUS

PENDING
AUTHORIZED
SETTLED
ERROR

AUTH_OUTCOME

SUCCESS
INSUFFICIENT_FUNDS
INCORRECT_DETAILS
SERVER_ERROR

SETTLEMENT_OUTCOME

SUCCESS
EXCEEDS_AUTH

D. Environment Variables

Variable	Description	Example
PORT	Backend port	3001
DB_PATH	SQLite file location	./database.db
PROVIDER_BASE_URL	Beeceptor API root	https://capstoneproject.free.beeceptor.com

E. Example Development Workflow

- Start backend:

```
cd server  
npm run dev
```


- Start frontend:

```
cd frontend  
npm run dev
```

- Create an order in Checkout.
- Observe authorization response in UI and DB.
- Finalize order in Settlement UI.
- View results in Order History UI.
- Delete database.db to reset data.

F. Notes on Scalability

The system was built for local execution but can be expanded easily:

- Replace SQLite with MySQL or PostgreSQL.
- Deploy backend to AWS, Azure, or Heroku.
- Deploy frontend to Vercel or Netlify.
- Add authentication or API keys.
- Integrate with a real payment provider.
- Add pagination logic at the backend level for large datasets.

AI Use Disclaimer

During the development of this project, Group 10 used artificial intelligence tools, including OpenAI's ChatGPT (GPT-5 model), as a supplemental resource. These tools supported the team in the same manner as technical documentation, online tutorials, or reference materials.

AI assistance was used in the following ways:

- Clarifying project requirements and technical expectations based on the Manhattan Associates project description and the KSU Capstone guidelines.
- Helping draft, edit, and format sections of this document, including the Functional Requirements, Technical Requirements, System Testing and Results, User Guide, and Appendix.

- Assisting with troubleshooting and debugging by suggesting possible causes of errors or offering example code patterns for the team to evaluate and integrate.
- Providing explanations of technologies used in the project such as Node.js, Express, React, and SQLite.
- Offering architectural guidance and serving as a sounding board during design discussions.

All AI-supported contributions were reviewed, tested, and integrated by the members of Group 10.

The completed system reflects the team's decisions, technical work, and understanding of the subject matter.

AI tools functioned only as an auxiliary resource and not as a replacement for student effort, analysis, or implementation.